

CS 229: Linear regression & gradient descent

(Lecture 2)

1 

Essential takeaways

1. Linear regression models a continuous output as a linear combination of input features.
2. Training means finding parameters θ that minimize prediction error.
3. Least squares uses the sum of squared residuals as its cost.
4. Gradient descent repeatedly moves parameters against the cost gradient.
5. Learning rate selection determines whether training is fast, slow, or unstable.
6. Batch gradient descent uses all examples, stochastic gradient descent (SGD) uses one, mini-batches use small groups.
7. Linear regression has a convex cost function, so it has no suboptimal local minimum.
8. The design matrix allows the complete model & cost to be expressed compactly.
9. The normal equation directly solves ordinary least squares.

2 

Important notation

m : nb of training examples

n : nb of input features

$x^{(i)}$: input vector of the i th example

$x_j^{(i)}$: feature j of training example i

$y^{(i)}$: target value of training example i

θ_j : param associated with feature j .

θ : vector of all model params

$h_\theta(x)$: Prediction made using params θ .

training example i

feature j

5

3

Core Formulas

Hypothesis: $h_{\theta}(x) = \theta^T x$

Cost function: $J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta} x^{(i)} - y^{(i)})^2$

Gradient: $\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^m (h_{\theta} x^{(i)} - y^{(i)}) x_j^{(i)}$

Gradient-descent update: $\theta := \theta - \alpha X^T (X\theta - y)$

Normal equation: $\theta = (X^T X)^{-1} X^T y$

4

Supervised-learning workflow

The general workflow is:

Training set $\rightarrow$ Learning algo $\rightarrow$ Hypothesis h_{θ}

The trained hypothesis then processes a new input

$x \rightarrow h_{\theta}(x) \rightarrow$ predicted y

5

Linear regression hypothesis

note:
sometimes θ_0 is written as b (bias) and $\theta_1, \dots, \theta_n$ as $w_1, \dots, w_n$ (weights)

Suppose we have 2 features: x_1 : house size x_2 : nb of bedrooms

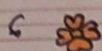
Hypothesis is: $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$ ($x_0 = 1$)

intercept
bias

influence of house size

influence of nb bedrooms

$h_{\theta}(x) = \sum_{j=0}^n \theta_j x_j$ or in vector form $h_{\theta}(x) = \theta^T x$ $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$ $x = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$



The cost function

- * We need a way to measure how well the hypothesis fits the training data.
- * Prediction error (residual) for one example: $h_{\theta}(x^{(i)}) - y^{(i)}$

- * Least squares cost function across the training set:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

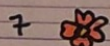
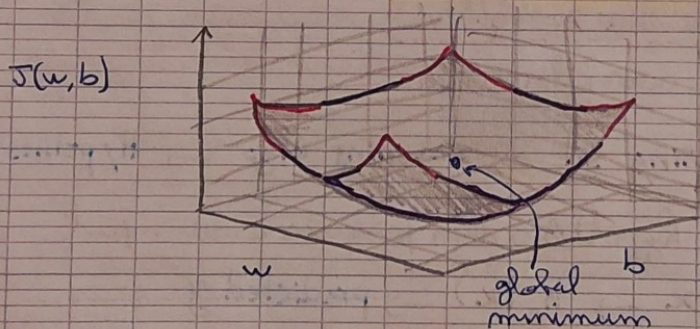
the $\frac{1}{2}$ just helps with derivation later

We square because it prevents + & - errors from cancelling out, emphasize large mistakes. Also has a statistical reason

note There are other types of cost functions.

- * So the learning objective is $\min_{\theta} J(\theta)$ (minimize $J(\theta)$ with respect to θ)

- * The cost function above look something like that:



Gradient descent

- * An iterative algo for minimizing $J(\theta)$.
- * Basically way to find path to the minimum is the figure above.
- * General update rule: $\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$

α : learning rate

gradient

after derivation: $\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$

or vector form:

$$\theta = \theta - \alpha X^T (X\theta - y)$$

8 Batch gradient descent & stochastic gradient descent* Batch gradient descent:

- The update uses the entire training set before changing θ .

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta} x^{(i)} - y^{(i)}) x_j^{(i)}$$

- Advantages:

- 1) Uses the exact gradient of the complete cost function.
- 2) Moves smoothly towards the min.
- 3) Works well with small/moderate data sets.
- 4) Easier to monitor/debug.

- Disadvantages:

Every single update requires processing all m examples, expensive.

* Stochastic gradient descent:

- SGD updates params using 1 training example at a time.

$$\theta_j := \theta_j - \alpha (h_{\theta} x^{(i)} - y^{(i)}) x_j^{(i)}$$

- Cheap, path towards minimum is noisy.
- With a constant learning rate, generally oscillates around the min. So early training was larger steps (α), while later smaller steps.

* Mini-batch gradient descent:

- Compromise between the 2 options above.
- Uses small groups (32, 64 or 128) examples per update.
- Usually the modern approach.

9

Matrix representation

(design matrix)

* Matrix of training examples:

$$X = \begin{bmatrix} x^{(1)T} \\ x^{(2)T} \\ \vdots \\ x^{(m)T} \end{bmatrix}$$

$\mathbb{R}^{m \times (n+1)}$

same as

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix}$$

since

$$x^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix}$$

* Target vector:

$$y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

* Parameter vector:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

* Predicted values matrix:

$$X\theta = \begin{bmatrix} \theta_0 + \theta_1 x_1^{(1)} + \dots + \theta_n x_n^{(1)} \\ \theta_0 + \theta_1 x_1^{(2)} + \dots + \theta_n x_n^{(2)} \\ \vdots \\ \theta_0 + \theta_1 x_1^{(m)} + \dots + \theta_n x_n^{(m)} \end{bmatrix}$$

$$= \begin{bmatrix} h_\theta(x^{(1)}) \\ h_\theta(x^{(2)}) \\ \vdots \\ h_\theta(x^{(m)}) \end{bmatrix}$$

* Residual vector:

(prediction errors)

$$r = X\theta - y = \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ h_\theta(x^{(2)}) - y^{(2)} \\ \vdots \\ h_\theta(x^{(m)}) - y^{(m)} \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{bmatrix}$$

* Cost function matrix notation:

So we know $J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}^{(i)} - y^{(i)})^2$

There is the property that $z^T z = \text{sum of squared elements of } z$

↳ The elements of $X\theta - y$ are $\sum_{i=1}^m (h_{\theta}^{(i)} - y^{(i)})^2$
 squared sum of

so $\Rightarrow J(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y)$

10

Matrix derivatives & trace notation

• Trace: For a square matrix A , trace is sum of diagonal elements:

$$\text{tr}(A) = \sum_i A_{ii}$$

• Important properties:

$$\text{tr}(A) = \text{tr}(A^T)$$

$$\text{tr}(AB) = \text{tr}(BA)$$

$$\text{tr}(ABC) = \text{tr}(CAB) \quad (\text{cyclic permutation})$$

$$\nabla_A \text{tr}(AB) = B^T$$

More about gradient ∇_A :

* ∇_A essentially means "calculate how result changes when each individual element of matrix A changes."

* ∇_A reads as "gradient with respect to A ."

* From an ordinary derivative to a gradient:

- If a function depends on several variables: $f(x, y) = x^2 + y^2$

- We need derivative for each variable: $\frac{\partial f}{\partial x} = 2x$ $\frac{\partial f}{\partial y} = 2y$

- The gradient collects these derivatives: $\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$

* Gradient with respect to a matrix:

- The gradient will contain one derivative for every element of A :

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \rightarrow \nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial a_{11}} & \frac{\partial f}{\partial a_{12}} \\ \frac{\partial f}{\partial a_{21}} & \frac{\partial f}{\partial a_{22}} \end{bmatrix}$$

11

The normal equation

- * Gradient descent finds the optimum iteratively.
- * For ordinary linear regression, it is also possible to solve for the optimum directly.

* Starting with $J(\theta) = \frac{1}{2} (\underbrace{X\theta - y})^T (X\theta - y) = \frac{1}{2} r^T r$

$$= \frac{1}{2} \sum_{i=1}^m r_i^2$$

reminder:
 $r^T r = \sum \text{squared elements of } r$

↳ First calculate the derivative with respect to one param θ_j . (chain rule)

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \left(\frac{1}{2} \sum_{i=1}^m r_i^2 \right)$$

derivative of $\frac{1}{2} r_i^2$ is $r_i \frac{\partial r_i}{\partial \theta_j}$

$$= \sum_{i=1}^m r_i \frac{\partial r_i}{\partial \theta_j}$$

$$= \sum_{i=1}^m r_i x_j^{(i)}$$

$$\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^m x_j^{(i)} (h_{\theta} x^{(i)} - y^{(i)})$$

we know $r_i = h_{\theta} x^{(i)} - y^{(i)}$

$$= \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots + \theta_n x_n^{(i)} - y^{(i)}$$

$$\Rightarrow \frac{\partial r_i}{\partial \theta_j} = x_j^{(i)}$$

↳ The gradient contains derivative for every param: $\nabla_{\theta} J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix}$

This looks like product of matrix multiplication

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \quad r = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{bmatrix}$$

$$X^T r$$

$$= \begin{bmatrix} \sum_{i=1}^m x_0^{(i)} r_i \\ \sum_{i=1}^m x_1^{(i)} r_i \\ \vdots \\ \sum_{i=1}^m x_n^{(i)} r_i \end{bmatrix}$$

$$\nabla_{\theta} J(\theta) = X^T r$$

$$\nabla_{\theta} J(\theta) = X^T (X\theta - y)$$

↳ At minimum, gradient = 0 $\Rightarrow X^T (X\theta - y) = 0$

$$X^T X \theta - X^T y = 0 \Rightarrow X^T X \theta = X^T y$$

if $X^T X$ invertible $\Rightarrow \theta = (X^T X)^{-1} X^T y$